

CV_Assignment2

전설민

May 2026

Contents

1	MSELoss vs CrossEntropy	2
2	Sigmoid vs Relu vs LeakyRelu	7
3	Optimizer	10

1 MSELoss vs CrossEntropy

MSE는 모든 경우에 대해 오차제곱합의 평균을 반환한다.

$$MSE = \frac{1}{n} \sum (y - \hat{y})^2$$

제공의 특성상 ($y - \hat{y}$)가 1보다 클 수록 극대화되지만 1보다 작으면 반대로 줄어든다.

하지만 분류 문제에서 MSE는 Softmax를 사용해야 한다.

분류는 답이 $[0, 0, \dots, 1, 0, 0, \dots, 0]$ 과 같이 0과 1의 확률형태로 주어지기 때문에 Softmax를 통해 예측값의 비중에 따라 확률의 총합이 1이 되도록 분배해야 한다. Softmax로 인해 분류 문제는 개별 오차가 1이 넘는 일이 없기 때문에 제공이 불리한 방향으로 작용한다.

그와 대비되게 CrossEntropy는

$$CE = -\sum y_{tar} * \log y_{pred}$$

위 형태로 주어지는데, y_{tar} 의 경우 분류문제에선 0과 1로 주어지므로 실질적으로 0에 해당하는 y_{tar} 의 오차는 사라지고 1에 해당하는 오차만 남는다.

그리고, $y_{tar} = 1$ 일 경우 $CE_i = \log y_{pred}$ 의 형태가 된다.

위 식과 로그의 특성으로 인해 타겟이 1일때 예측치가 0에 가까울 경우 $\log y_{pred}$ 는 음의 무한대로 발산하며, CE앞에 붙은 마이너스로 인해 총 값은 양의 무한대로 발산한다.

요약: MSE는 아무리 예측이 틀려도 1을 넘지 못한다.

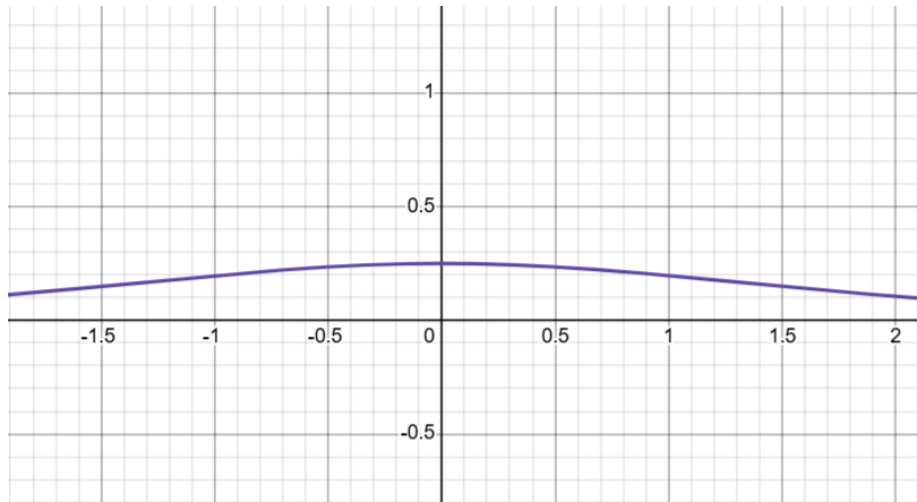
그에비해 CrossEntropy는 예측이 틀릴수록 극적으로 무한대로 발산한다.

이 차이로 인해 MSELoss는 미분시 기울기가 작다.

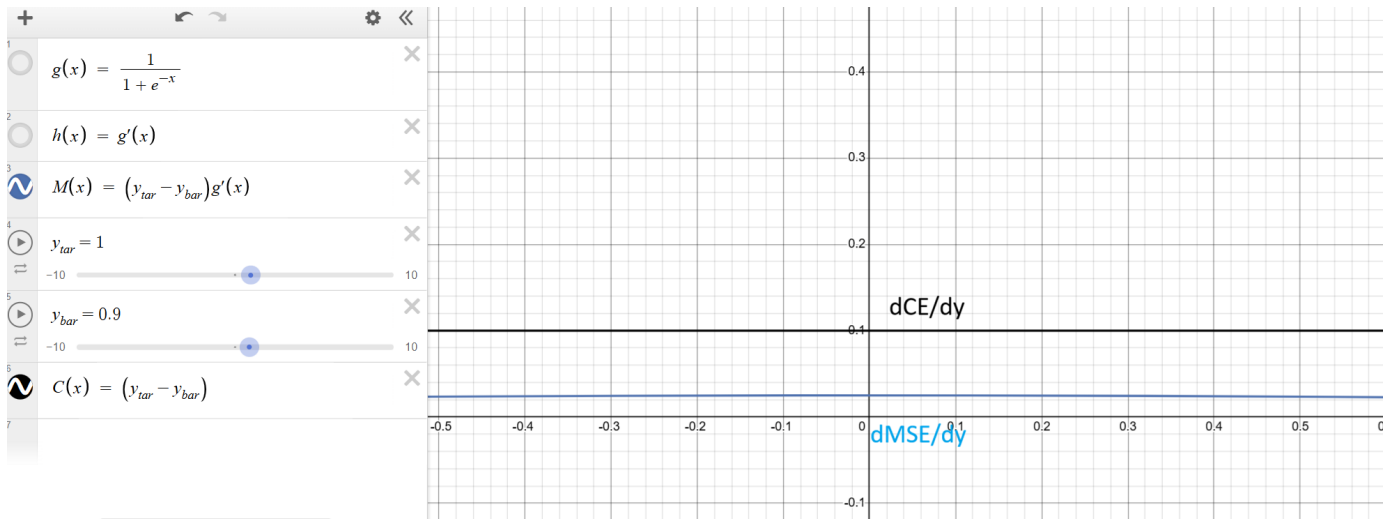
미분시 기울기가 작다는 것은 층을 거듭할수록 기울기가 빠르게 소실되는 것을 의미하며 학습효율도 기울기의 미분을 기반으로 하기 때문에 MSE쪽이 나쁘다.

또한, 시그모이드를 사용한 MSELoss는 미분할 경우 시그모이드가 남는다.

이는 시그모이드 특유의 경사 소실문제와 MSE의 작은 값이 역시너지를 이뤄 경사소실 문제를 심화시킨다.



시그모이드의 도함수 σ' 는 위와 같이 1을 넘지 못하며 0과 멀어질수록 작아진다.
 CrossEntropy는 미분과정에서 시그모이드 미분과 상쇄되어 사라지므로
 $\frac{dCE}{dy_{pred}} = (y_{pred} - y_{tar})$ 와 같이 시그모이드가 없지만
 MSE의 경우는
 $\frac{dMSE}{dy_{pred}} = (y_{pred} - y_{tar})\sigma'(z)$ 형태로 시그모이드가 남으므로
 실질 기울기가 줄어들어 경사소실에 취약해진다

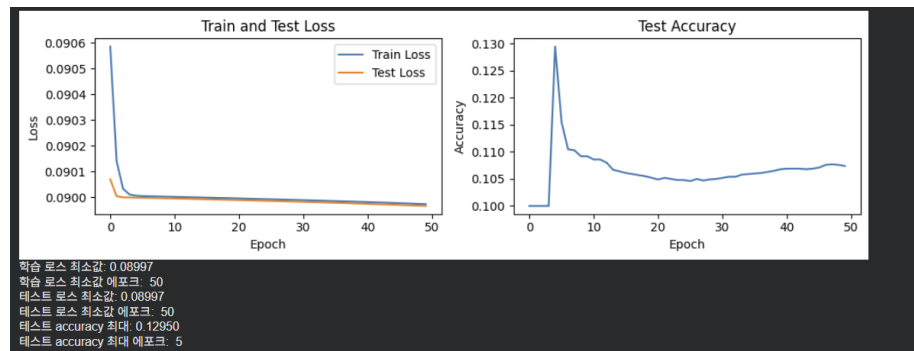


위와 같이 같은 y_{tar}, y_{pred} 임에도 차이가 큰 것을 볼 수 있다.

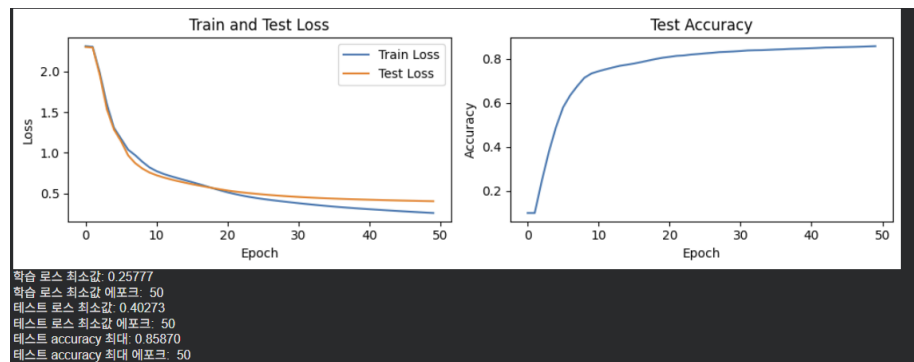
에포크: 50, 학습률: 0.05, 배치크기: 64, 시그모이드 사용
 신경망 구조 : 입력(28 * 28) → 128 → 64 → 64 → 출력(10)

에포크 당 로스와 accuracy

MSE



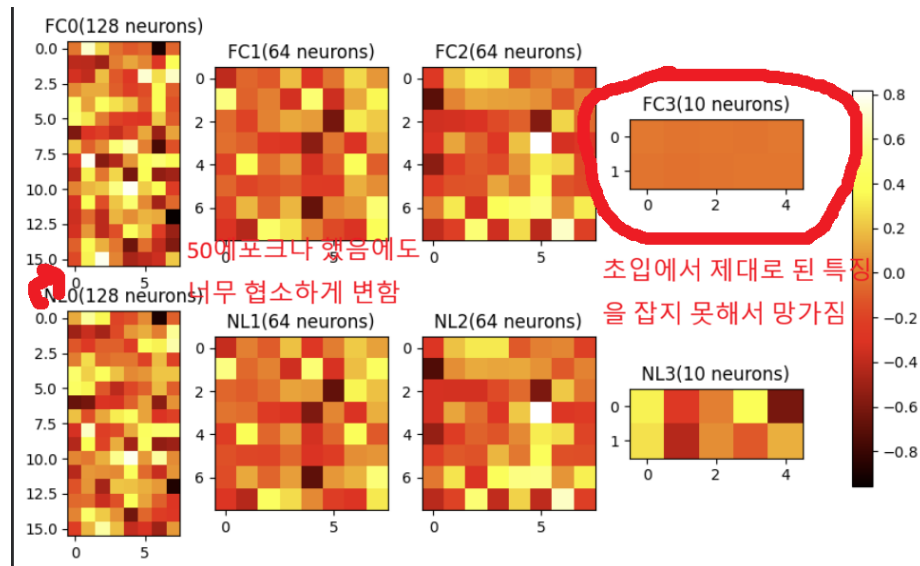
CrossEntropy



위 그래프를 보면, Loss값은 시스템이 다르므로 직접 비교하기 어려우나,
 CrossEntropy쪽 Loss값이 비율상 빠르게 감소하며, Accuracy는 현저한 차이를 보인다.

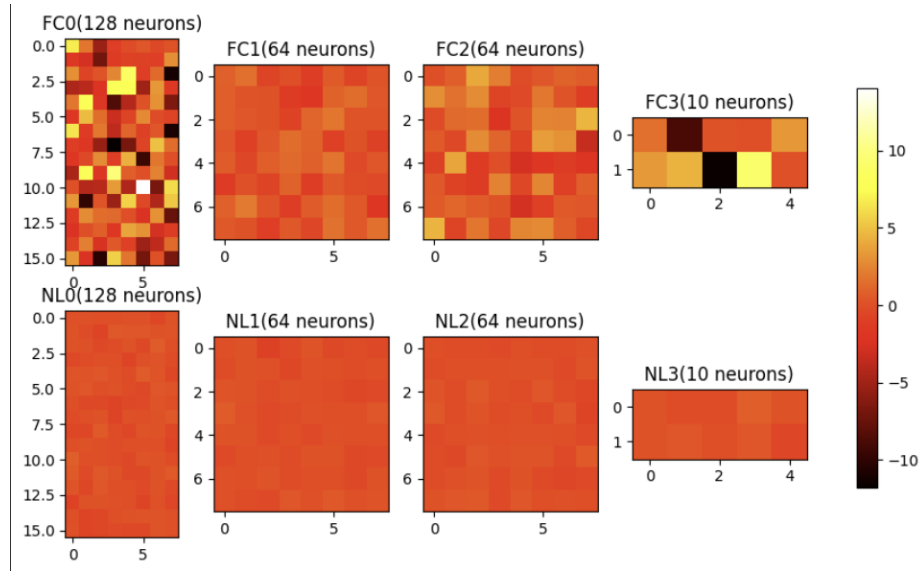
히트맵(FC는 학습 후, NL은 학습 전)
 동일 logits를 입력해서 학습전과 학습후 각 노드의 활성화 함수가 어떻게 작동하는지 분석했다

MSE



위 그래프를 보면 1레이어에 역전파를 통해 특징을 잡아야 하는데 오차율에 비해 변화가 지나칠 정도로 협소하다. 원인은 3층에서 1층까지 전달할 MSE값이 충분히 크지 않은 문제, 시그모이드 함수의 경사 소실 문제. 해당 두 문제가 겹쳐서 학습이 원활하지 않기 때문이다.

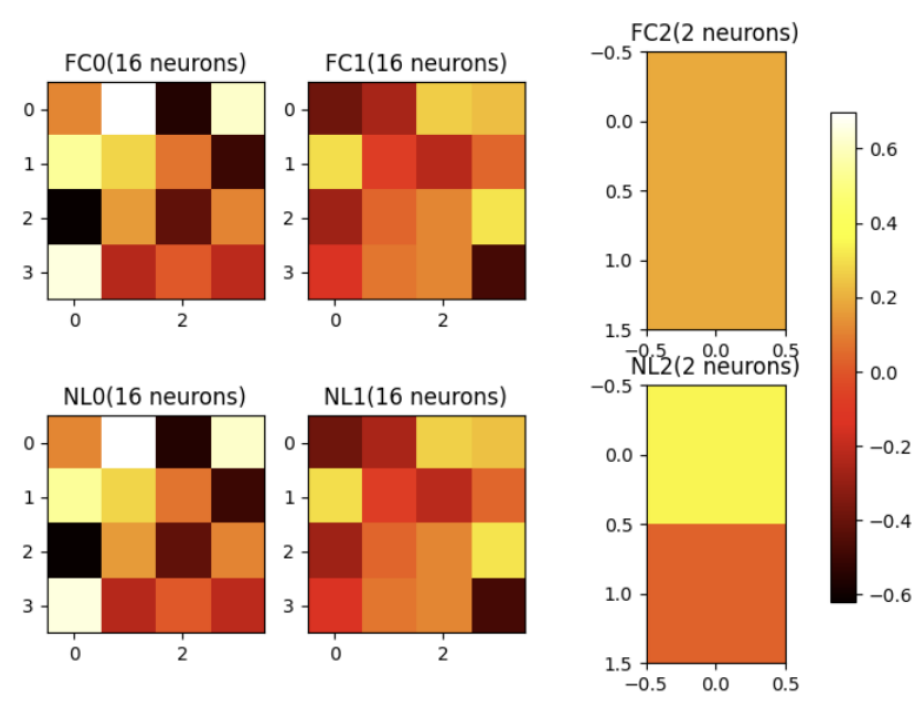
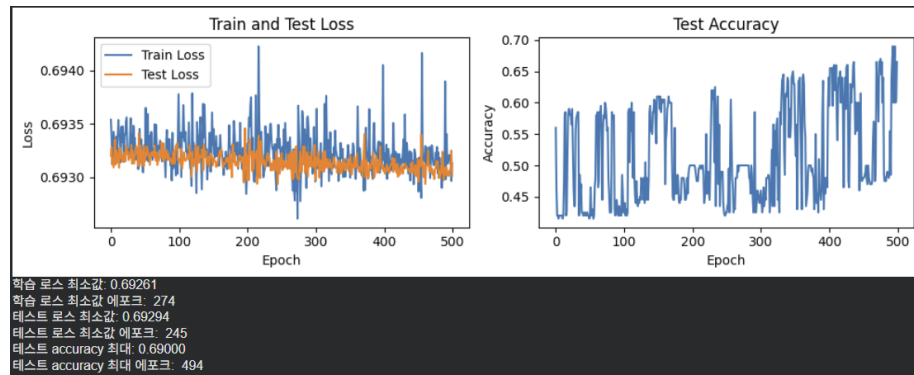
CrossEntropy



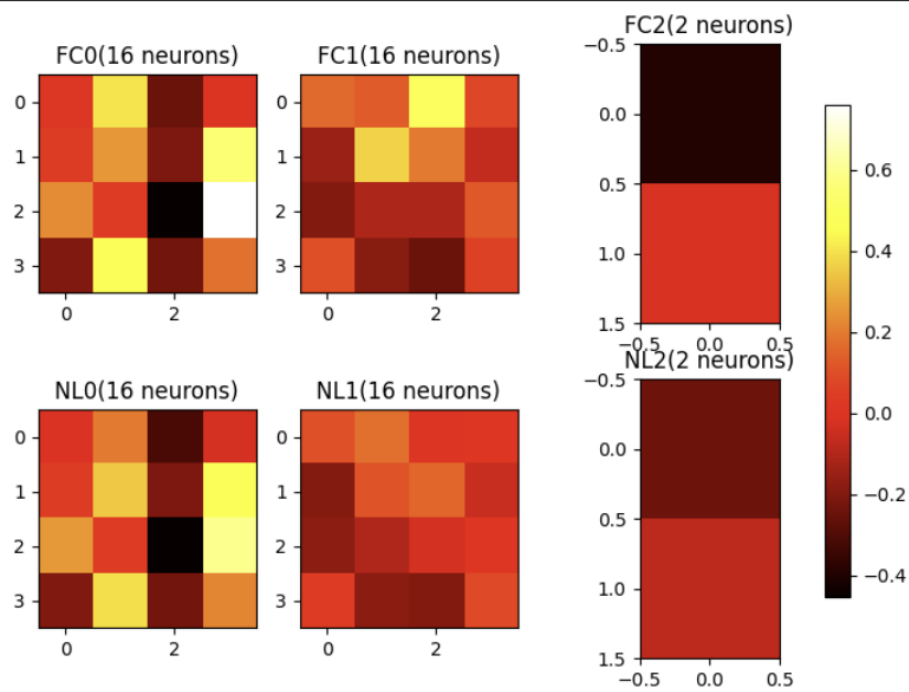
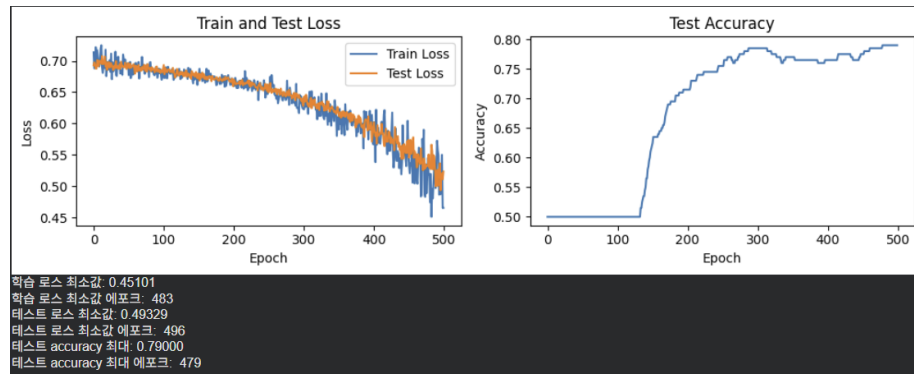
그에 비해 같은 조건에서 CrossEntropy를 사용한 뉴럴 네트워크의 히트맵을 보면 첫 레이어가 매우 극명한 특징을 학습하는데 성공한 것을 볼 수 있다.

2 Sigmoid vs Relu vs LeakyRelu

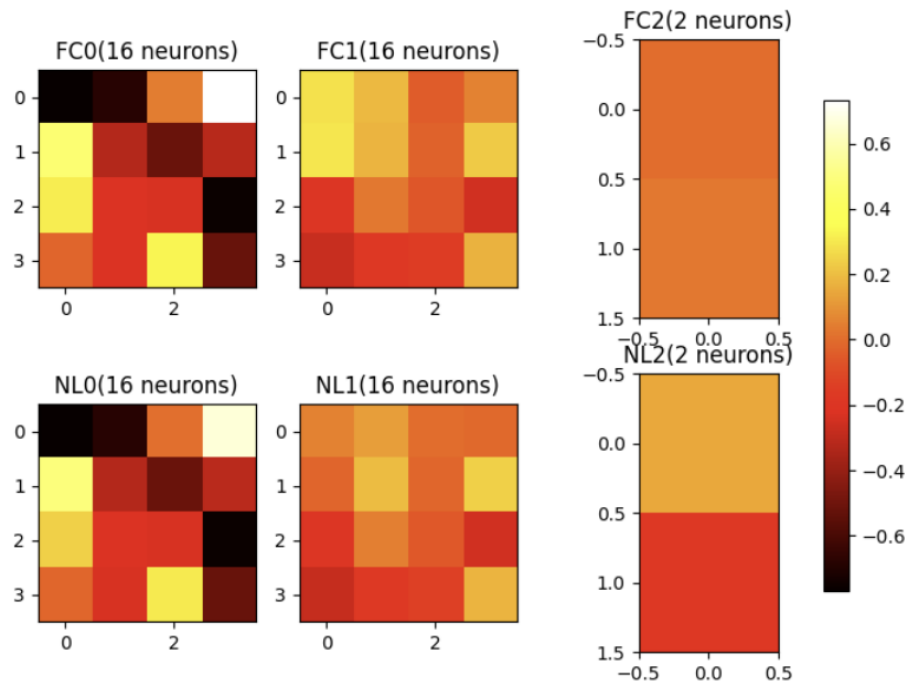
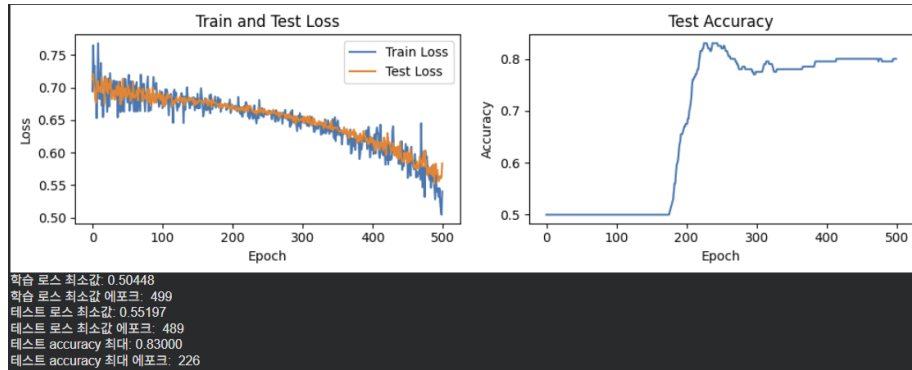
Sigmoid



Relu



LeakyRelu



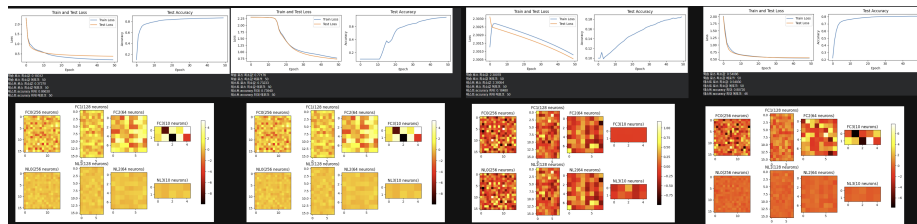
시그모이드는 3페이지의 도함수에 따라 경사 소실이 일어나며, ReLU 또한 $\max(x, 0)$ 로 x 가 음수일 경우 경사가 소실된다. 그에 비해 LeakyReLU는 $\max(x, 0.1x)$ 으로 x 가 음수일 경우에도 기울기가 0.1로 소실되지 않으므로 Dead ReLU를 완화할 수 있다.

3 Optimizer

신경망 구조 : 입력($28 * 28$) $\rightarrow$ 256 $\rightarrow$ 128 $\rightarrow$ 64 $\rightarrow$ 출력(10)
에포크: 50, 배치크기: 64, 시그모이드 사용, CrossEntropy 사용

각 최적화 함수를 0.1, 0.01, 0.001의 러닝 레이트로 학습시켜봤다.

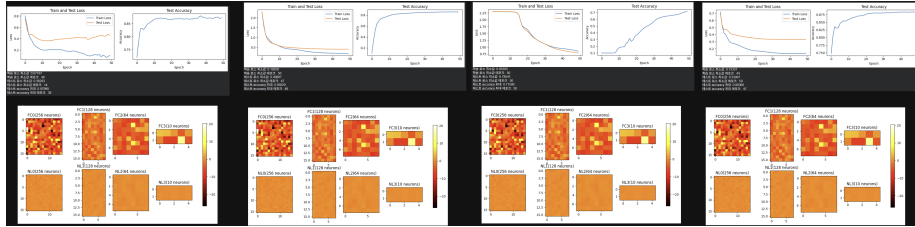
1. SGD



- 결과요약:

왼쪽부터 순서대로 각각 0.1, 0.01, 0.001, 0.1+스케줄링(감마=0.9)이다.
SGD에서는 0.1이 가장 빠르게 수렴하고, 높은 정확도를 보이는 학습률이다.
관성도, 학습률 조정도 없어서 Loss가 줄어들 수록 빠르게 수렴해버린다.
따라서 학습률이 낮아질수록 수렴속도가 느려지며 가장 낮은 0.001은
50에포크를 전부 시행해도 수렴하지도 못하는 모습이다.
0.1에 스케줄링을 부여한 결과 학습곡선이 완만해지긴 하지만 0.1도 부족한
상황에서 스케줄링으로 감산하는 것은 부적절해 보인다.

2. SGD+Momentum



- 결과 요약:

학습률 배치 순서는 위 SGD와 같다.

SGD의 수식을 다음과 같이 가정한다.

$$w_{n+1} = w_n - \alpha \frac{dL}{dw_n} = w_n - v_t$$

그럴 경우 SGD+Momentum의 식은 다음과 같이 정리 가능하다.

$$v_n = \gamma v_{n-1} - \alpha \frac{dL}{dw_n}$$

$$w_{n+1} = w_n - v_n$$

즉 가중치 w 에 대한 변위 v_t 가 누적되는 형태를 가진다.

이는 기울기가 이전과 같은 방향성을 가질 경우 누적되어 학습이 가속한다.

그리고, 기울기가 이전과 다른 방향성을 가질 경우 상쇄되어 학습이 느려진다.

이를 통해 경향에 따라 학습률을 유연하게 조정하기에 위 2번, 3번 그림처럼

SGD만으로는 수렴하는데 많은 에포크가 필요한 학습률을 가진 모델도

가속이 누적되어 안정적인 학습을 보여준다.

하지만 $v_t = \gamma v_{t-1} + \dots$ 과 같이 v_t 는 감마를 비로 하는 수열의 성질도 가지고 있다.

만일 감마를 0.9라 가정하면, $0.9^8 \approx 0.43$.

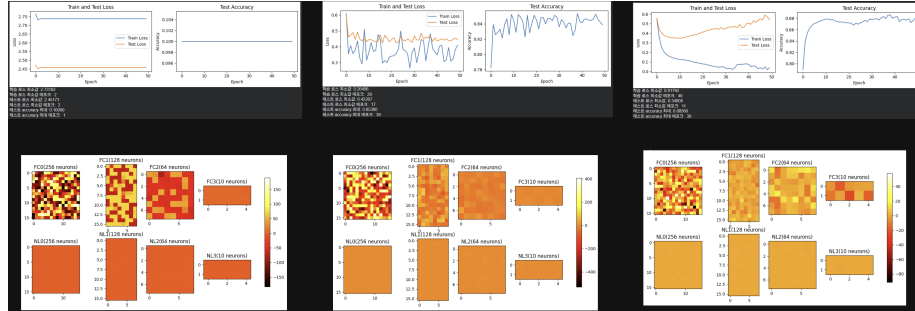
다시 말해 이전 학습의 영향에 대한 반감기가 8회에 달한다.

따라서 1번 그림처럼 학습률이 높을 경우 관성으로 인한 Overshoot이 발생 가능하다.

따라서 학습률을 낮추거나, 에포크 수를 줄여 과적합을 막을 필요가 있다.

혹은 4번처럼 점진적으로 감소하는 스케줄을 통해 오버핏을 제어하는 방법도 있다.

3. Adam



- 결과 요약:

1번 그림: 0.1은 학습률이 지나치게 커서 학습이 전혀 진행이 안되는 모습이다.
 학습률 0.01는 학습을 진행하긴 하나, 여전히 커서 제대로 수렴하지 못한다.
 학습률 0.001이 되어서야 SGD+Momentum의 학습률 0.1그림처럼 진동한다.